Software Engineering Assignment

Task 5 : CI Pipeline

Subject: Software Engineering (24PCA502), BCA 5th Semester
Name: [STUDENT NAME / Sandeep Kharwar / Rahul S H] **USN:** [STUDENT USN / NU24UCA063]
GitHub repository: https://github.com/Rahul-S-H-7/SE-TASK

1. Introduction

This assignment implements a **Smart Attendance Management System** with an automated Continuous Integration (CI) pipeline. In higher education, maintaining student attendance records and identifying exam eligibility based on mandatory regulatory quotas is a critical administrative activity. This system provides student registration, class session scheduling, and attendance recording with robust duplicate prevention.

The standalone business module attendance_logic.py computes attendance percentages and enforces exam eligibility rules. A GitHub Actions CI workflow triggers on every push and pull request to execute automated pytest tests, immediately catching regressions.

Business rules implemented in attendance_logic:

- **Percentage Calculation:** Attendance % = (Classes Attended / Conducted Sessions) × 100, rounded to 2 decimal places.
- **Zero Classes Boundary:** If 0 sessions have been conducted, attendance is strictly 0.0% and the student is ineligible.
- **75% Eligibility Quota:** Exactly ≥ 75.0% attendance qualifies as eligible; < 75.0% is strictly ineligible.
- **Data Integrity & Range:** Valid percentages remain between 0.0% and 100.0%. Negative numbers or attended > conducted raise ValueError.
- **Duplicate Prevention:** Each student can have only one attendance record per session, enforced via SQLite UNIQUE constraints.

2. Implementation (attendance_logic.py)

```
ELIGIBILITY_THRESHOLD = 75.0

def calculate_attendance(attended_classes: int, conducted_classes: int) -> dict:
    if not isinstance(attended_classes, int) or not isinstance(conducted_classes, int):
        raise TypeError("attended_classes and conducted_classes must be integers")
    if attended_classes < 0:
        raise ValueError("attended_classes cannot be negative")
    if conducted_classes < 0:
        raise ValueError("conducted_classes cannot be negative")
    if attended_classes > conducted_classes:
        raise ValueError("attended_classes cannot exceed conducted_classes")
    if conducted_classes == 0:
        return {"attended": 0, "total": 0, "percentage": 0.0, "is_eligible": False}

    percentage = round((attended_classes / conducted_classes) * 100.0, 2)
    is_eligible = percentage >= ELIGIBILITY_THRESHOLD
    return {
        "attended": attended_classes, "total": conducted_classes,
        "percentage": percentage, "is_eligible": is_eligible
    }
```

3. Tests (tests/test_attendance.py)

A comprehensive test suite of 19 automated tests was written with pytest, using isolated temporary SQLite database fixtures to verify calculation correctness, boundary conditions, and database constraints:

```
import pytest
from attendance_logic import calculate_attendance, validate_status

def test_zero_classes_conducted():
    assert calculate_attendance(0, 0)["percentage"] == 0.0

def test_100_percent_attendance():
    assert calculate_attendance(20, 20)["is_eligible"] is True

def test_exactly_75_percent_boundary():
    assert calculate_attendance(3, 4)["is_eligible"] is True

def test_just_below_75_percent_ineligible():
    assert calculate_attendance(74, 100)["is_eligible"] is False

def test_correct_percentage_rounding():
    assert calculate_attendance(2, 3)["percentage"] == 66.67

def test_duplicate_attendance_prevention(temp_db):
    st_id = db.add_student("NU24UCA001", "Alice", "BCA 5th Sem", db_path=temp_db)
    ses_id = db.create_session("2026-10-26", "SE", "BCA 5th Sem", db_path=temp_db)
    db.mark_attendance(ses_id, st_id, "Present", db_path=temp_db)
    with pytest.raises(ValueError, match="already recorded"):
        db.mark_attendance(ses_id, st_id, "Present", db_path=temp_db)
```

4. GitHub Actions Workflow (.github/workflows/ci.yml)

```
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - run: pytest -v
```

How it works: The workflow automatically triggers on every push and pull request. It provisions an Ubuntu runner, checks out the repository, installs Python 3.12 with dependencies from requirements.txt (Flask, pytest), and executes pytest -v. If any test fails, the CI run immediately exits with failure, preventing defective code from advancing.

5. CI Results

A. Failing run demonstration

To verify that the continuous integration pipeline actively catches defects before deployment, an intentional bug was introduced in attendance_logic.py on branch demo/intentional-defect (Commit: 78f3421). The eligibility condition was changed from percentage >= ELIGIBILITY_THRESHOLD to strictly percentage > ELIGIBILITY_THRESHOLD. This broke the critical 75.0% boundary condition where a student with exactly 75% attendance was incorrectly marked as ineligible.

```
$ python -m pytest -v
============================= test session starts ==============================
tests/test_attendance.py::test_zero_classes_conducted PASSED             [  5%]
tests/test_attendance.py::test_100_percent_attendance PASSED             [ 10%]
tests/test_attendance.py::test_exactly_75_percent_boundary FAILED        [ 15%]
tests/test_attendance.py::test_just_below_75_percent_ineligible PASSED   [ 21%]
tests/test_attendance.py::test_independent_calculations_multiple_students FAILED [100%]
================================== FAILURES ===================================
_________________________ test_exactly_75_percent_boundary _____________________
    def test_exactly_75_percent_boundary():
        result = calculate_attendance(3, 4)
        assert result["percentage"] == 75.0
>       assert result["is_eligible"] is True
E       assert False is True
======================== 2 failed, 17 passed in 0.26s ==========================
```



Figure 1: GitHub Actions Run #5 (failure) triggered by intentional 75% boundary defect (Commit 78f3421).

B. Passing run restoration

The defect was resolved on branch demo/failing-ci (Commit: dc54adf) by restoring the inclusive comparison percentage >= ELIGIBILITY_THRESHOLD. All 19 tests in the test suite passed cleanly both locally and on the GitHub Actions runner (Workflow Run #4), achieving 100% test success.

```
$ python -m pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
collected 19 items

tests/test_attendance.py::test_zero_classes_conducted PASSED             [  5%]
tests/test_attendance.py::test_100_percent_attendance PASSED             [ 10%]
tests/test_attendance.py::test_exactly_75_percent_boundary PASSED        [ 15%]
tests/test_attendance.py::test_just_below_75_percent_ineligible PASSED   [ 21%]
tests/test_attendance.py::test_correct_percentage_rounding PASSED        [ 26%]
tests/test_attendance.py::test_boundary_just_above_75_percent PASSED     [ 31%]
tests/test_attendance.py::test_invalid_negative_attended_classes PASSED  [ 36%]
tests/test_attendance.py::test_invalid_negative_conducted_classes PASSED [ 42%]
tests/test_attendance.py::test_attended_exceeds_conducted_classes PASSED [ 47%]
tests/test_attendance.py::test_invalid_types_raise_type_error PASSED     [ 52%]
tests/test_attendance.py::test_validate_status_valid_values PASSED       [ 57%]
tests/test_attendance.py::test_validate_status_invalid_value PASSED      [ 63%]
tests/test_attendance.py::test_database_persistence_and_student_addition PASSED [ 68%]
tests/test_attendance.py::test_student_unique_id_constraint PASSED       [ 73%]
tests/test_attendance.py::test_duplicate_attendance_prevention PASSED    [ 78%]
tests/test_attendance.py::test_nonexistent_student_attendance_rejected PASSED [ 84%]
tests/test_attendance.py::test_nonexistent_session_attendance_rejected PASSED [ 89%]
tests/test_attendance.py::test_missing_student_details_rejected PASSED   [ 94%]
tests/test_attendance.py::test_independent_calculations_multiple_students PASSED [100%]
============================== 19 passed in 0.22s ==============================
```



Figure 2: GitHub Actions Runs #4 & #3 (success) demonstrating passing test suite on restored code.

6. Conclusion

The automated Continuous Integration pipeline runs the comprehensive unit and integration test suite automatically on every Git push and pull request. Any accidental regression—such as changing an attendance threshold or mishandling edge cases—is caught within seconds on the GitHub Actions runner before defective code can reach production or corrupt student academic standing records. This guarantees consistent software reliability, traceability, and high engineering quality.